UNIVERSITY OF SCIENCE

VNU-HCM

FACULTY OF INFORMATION TECHNOLOGY

PROJECT DATABASE PHASE 1

Project Title: CAMPUS SPACE MANAGEMENT SYSTEM

Course: Introduction to Database System - CS486

Class: 24A02

Submitted by:
- Trần Đức Thiên (ID: 24125081)
- Lê Đức Thịnh (ID: 24125045)
- Trịnh Văn Thành (ID: 24125044)
- Nguyễn Hoàng Gia (ID: 24125099)

3rd July 2026

Contents

1 General Information

1.1 Project Directory Structure

The project repository is organized around a prompt-driven agent pipeline. The `.opencode/` directory contains all agent definitions, slash commands, evaluation rubrics, and output templates; the `outputs/` directory contains only the artifacts generated by running that pipeline and is never edited directly (see Section 2.3).

Project Directory Structure

```
|-- .opencode/
|   |-- agent/
|   |   |-- business-analyst.md
|   |   |-- conceptual-database-designer.md
|   |   |-- database-definition-implementation-engineer.md
|   |   |-- database-design-reviewer.md
|   |   |-- logical-database-designer.md
|   |   |-- sample-data-preparer.md
|   |   `-- sql-query-designer.md
|   |-- commands/
|   |   |-- analyze-requirement.md
|   |   |-- design-conceptual-database.md
|   |   |-- design-db.md
|   |   |-- design-logical-database.md
|   |   |-- design-query.md
|   |   |-- implement-database-definition.md
|   |   |-- prepare-sample-data.md
|   |   `-- validate-database-design.md
|   |-- evaluation/
|   |   |-- conceptual-design-rubric.md
|   |   |-- ddl-rubric.md
|   |   |-- logical-design-rubric.md
|   |   |-- query-design-rubric.md
|   |   |-- requirement-analysis-rubric.md
|   |   |-- sample-data-rubric.md
|   |   `-- validation-rubric.md
|   |-- skills/
|   |   `-- db-design-pipeline/
|   `-- templates/
|       |-- conceptual-design-template.md
|       |-- ddl-template.md
|       |-- logical-design-template.md
|       |-- query-design-template.md
|       |-- requirement-analysis-template.md
|       |-- sample-data-template.md
|       `-- validation-template.md
|-- AGENTS.md
|-- README.md
|-- outputs/
|   |-- 01-business-req-analysis-G03.md
|   |-- 02-erd-design-G03.md
|   |-- 03-logical-design-G03.md
|   |-- 04-design-validation-G03.md
|   |-- 05-db-definition-G03.sql
|   |-- 06-sample-data-G03.sql
|   `-- 07-query-design-G03.sql
`-- req/
    `-- business-requirement.md
```

Directory roles:

- `req/` — the original business requirement specification for the Campus Space Management System, used as the single source of truth for all downstream agents.
- `.opencode/agent/` — one prompt file per pipeline stage, defining that agent's rules, workflow, and output

format.

- `.opencode/commands/` — slash commands used to invoke each agent (or the full pipeline via `design-db.md`).
- `.opencode/evaluation/` — one rubric per stage, used for both human review and agent refinement (Section 2.2).
- `.opencode/templates/` — output templates that standardize the structure of each generated artifact.
- `outputs/` — the artifacts produced by running the pipeline; regenerated from `.opencode/` and never edited by hand.

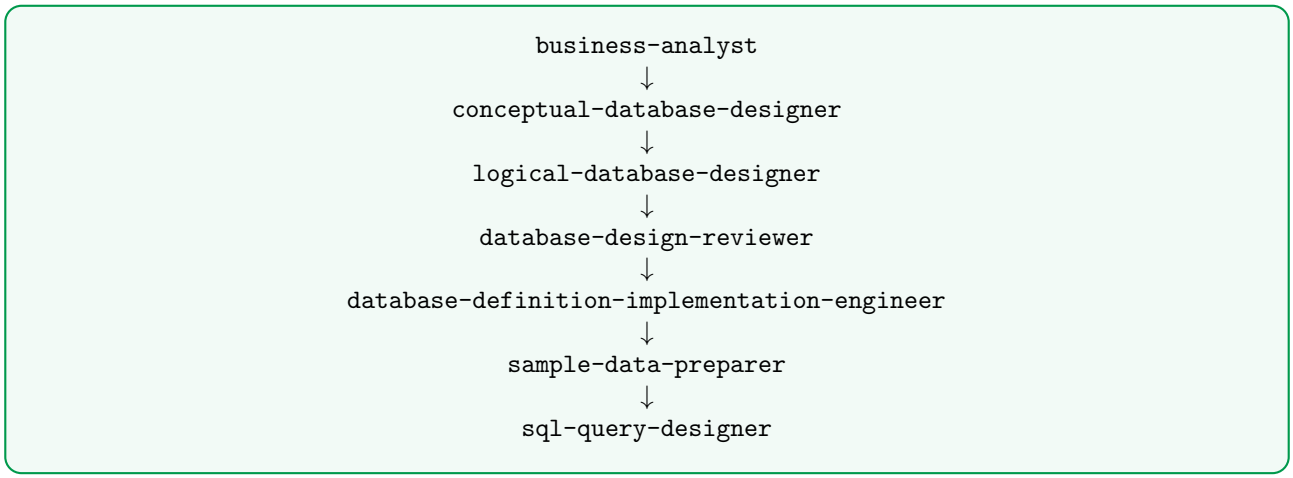
1.2 Model Used

All agents in the pipeline were run using **GPT - 5.5** via the OpenCode CLI[1]. The same model was used consistently across all seven agents and across all pipeline versions (Version 1–3) to ensure that observed improvements came from refinements to the agent prompts/rules rather than from changes in the underlying model.

2 Agent Improving Process

2.1 Workflow Overview

The project uses seven specialized agents stored in the `agent/` directory. They operate sequentially, where each agent receives the output of the previous agent as its input.



- `business-analyst.md`: extracts actors, entities, attributes, relationships, and business rules from the original specification.
- `conceptual-database-designer.md`: converts the requirement analysis into a conceptual ERD.
- `logical-database-designer.md`: transforms the conceptual design into a relational schema with keys and constraints.
- `database-design-reviewer.md`: reviews the schema for correctness, consistency, and compliance with the requirements.
- `database-definition-implementation-engineer.md`: implements the approved schema as SQL DDL.
- `sample-data-preparer.md`: generates realistic sample data for normal and exceptional cases.
- `sql-query-designer.md`: creates meaningful SQL queries based on the implemented database.

This sequential workflow preserves traceability from the original requirements to the final SQL queries and allows detected errors to be traced back to the responsible agent.

[1] https://opencode.ai

2.2 Evaluation Methodology

The project adopted a ***human-in-the-loop*** evaluation approach to assess and improve the agent-based workflow. All database artifacts were generated by specialized agents. The group members primarily acted as evaluators who reviewed the generated outputs and refined the agents, rather than manually completing or directly correcting the deliverables.

Human Review

Each member independently reviewed the artifacts stored in the `outputs/` directory. Every artifact was compared with the original *Campus Space Management System* specification to identify errors that might have been overlooked by the corresponding agent. The review focused on the following issues:

- Missing or incorrectly interpreted requirements;
- Unsupported assumptions or hallucinated information;
- Incorrect entities, attributes, relationships, or cardinalities;
- Inappropriate keys and database constraints;
- Duplicated information and violations of the Single Source of Truth principle;
- Inconsistencies between consecutive stages of the agent pipeline;

Each artifact was evaluated against both the original project specification and the output of the preceding agent. For example, the logical schema was checked against the conceptual design, while the SQL DDL was checked against the logical schema. This cross-stage review helped identify errors that were introduced in an earlier stage and propagated into later deliverables.

Rubric-Based Evaluation

To make the evaluation process systematic and repeatable, the group created an evaluation/ directory. It contains one rubric file for each stage of the agent pipeline. Each rubric was first built from a set of basic criteria derived directly from the stage's expected deliverables (e.g., correct cardinalities for the conceptual design, or named constraints for the DDL), and was then progressively extended with additional criteria whenever a new error or oversight was discovered during human review—so that the same mistake would be systematically checked for in every subsequent run of the pipeline.

- `requirement-analysis-rubric.md`

  Evaluates the business-requirement analysis, including business objectives, actors, entities, attributes, relationships, cardinalities, and business rules.

- `conceptual-design-rubric.md`

  Evaluates whether the conceptual ERD correctly represents the identified entities, attributes, relationships, cardinalities, and participation constraints.

- `logical-design-rubric.md`

  Evaluates the relational schema, including relations, primary keys, foreign keys, candidate keys, relationship mappings, and normalization decisions.

- `validation-rubric.md`

  Evaluates the output of the database-design reviewer agent. It checks whether the reviewer identifies actual design defects, avoids unsupported criticism, and distinguishes database-model issues from application-level logic.

- `ddl-rubric.md`

  Evaluates the SQL DDL implementation, including tables, data types, primary keys, foreign keys, `UNIQUE` constraints, `CHECK` constraints, default values, and referential actions.

The rubric files provided consistent evaluation criteria for both human reviewers and the agent improvement process. They also made it easier to compare the quality of outputs across different pipeline versions.

2.3 Improvement Strategy

The group followed the principle of improving the agents rather than manually correcting their generated artifacts.

Core Principle

Fix the source—prompts and rules—not the generated output.

Whenever a defect was identified, the group traced it back to the responsible agent. The issue was then converted into a clearer constraint and added to the relevant prompt, rule, template, rubric, or self-check instruction.

Evaluate Output → Identify Root Cause → Update `.opencode/`
→ Re-run the Pipeline → Verify the Result

This iterative process reduced repeated errors while ensuring that the final artifacts remained generated by the agent pipeline.

2.4 Key Refinement Iterations

The full pipeline consists of seven steps: four design steps (business analysis, conceptual design, logical design, and design validation) followed by three implementation steps (DDL implementation, sample-data generation, and query design).

Versioning Scheme. In this report, a version refers to one complete cycle of the "Evaluate → Identify Root Cause → Update .opencode/ → Re-run the Pipeline → Verify" process described in Section 2.3, applied to a defined subset of the seven pipeline agents. Each re-run overwrites the corresponding files in `outputs/`; the group did not retain separate `v1/`, `v2/`, `v3/` snapshot folders, so the "Verification results" reported at the end of each version subsection describe the state of outputs/ immediately after that version's pipeline re-run, before the next round of defects was identified. The three versions differ in scope, not in the files they touch — all three re-run the full seven-agent pipeline, but human review and agent refinement were deliberately staged:

- Version 1 and Version 2 restrict evaluation to the four design outputs (files 01–04: business analysis, conceptual design, logical design, validation), since defects at this stage propagate downstream and are the most costly to leave unresolved.
- Version 3 extends evaluation to the three implementation outputs (files 05–07: DDL, sample data, queries), once the design stage from Versions 1–2 had stabilized.

As explained above, Versions 1–2 cover files 01–04 and Version 3 extends coverage to files 05–07; see the corresponding subsections below

2.4.1 Version 1

Context and Evaluation Methodology The full pipeline consists of seven agents, but in Version 1 we ran and evaluated only the first four (the design steps), with each agent taking the output of the previous step as input:

1. business-analyst → `01-business-req-analysis-G03.md`
2. conceptual-database-designer → `02-erd-design-G03.md`
3. logical-database-designer → `03-logical-design-G03.md`
4. database-design-reviewer → `04-design-validation-G03.md`

The table below presents, for each agent: limitations found in Version 1 → root cause → changes made to the agent definition to fix them → evidence of remediation in the subsequent version.

Agent 1 — business-analyst (file 01) The Version 1 business requirements analysis went too deep and exceeded the intended scope: it touched logical-level details (mentioning foreign keys) whereas this step should remain at the business/conceptual level. Additionally, some content involved over-inference beyond the source, some information was duplicated across entities, and entire sections on processes (state transitions, role-based permissions, workflows, and cross-entity constraints) were missing.

Table 1: Summary of Business Analyst Refinements

Issue Categories	Examples in Version 1	Refinements (Agent/Prompt Changes)
1. Hallucination & Over-engineering	• Fabricated a `usage_policy` check rule that was not in the business requirements. • Listed redundant Actors (e.g., "Staff") based solely on the introduction. • Added implicit attributes without proper traceability.	• Added **Rule 1 (Source-grounding)**: Strictly prohibit fabricating rules; all constraints must trace back to the source document. • Added **Rule 1.1**: Require `[proposed/inferred]` tags for logically derived elements. • Added **Rules 5 & 7**: Deduplicate and filter out actors without actual responsibilities.
2. Violation of Database Principles	• `rejection_reason` duplicated in both Booking Request and Approval Decision. • Merged independent actions (e.g., check-in and complete session) into a single relationship.	• Added **Rule 3 (Single source of truth)**: Ensure each fact/event is attached to only one entity. • Added **Rule 4 (Distinct actions)**: Explicitly separate independent actions into distinct relationships to prepare for conceptual design mapping.
3. Scope Confusion & Template Incompleteness	• Attempted to handle logic which is not clearly stated in the requirement • Missing crucial descriptions for State transitions, Role permissions, and Workflows.	• **Template Update**: Added 4 mandatory sections (State Transitions, Workflow, Role Permissions, Cross-Entity Constraints). • Added **Rule 6** : Instructed the agent to write the unclear informations, logic in "Open Question" section

Agent 2 — conceptual-database-designer (file 02) The Version 1 ERD inherited errors from file 01 (missing tag for decision outcome, duplicate `rejection_reason`) and introduced its own presentation issues: merged relationships, cardinality symbols that did not distinguish mandatory/optional participation, mismatched relationship labels between the Mermaid diagram and §4 table, and an invented attribute (`facility_description`).

Table 2: Summary of Conceptual Designer Refinements

Issue Categories	Examples in Version 1	Refinements (Agent/Prompt Changes)
1. Tool Limitation Hallucinations	• The agent falsely claimed Mermaid.js could not draw multiple lines between the same entity pair, using this to wrongfully merge independent relationships.	• Added **Rule 7 (One line per relationship)**: Explicitly banned this incorrect technical justification. • Required the number of diagram lines to strictly match the relationship tables.
2. Notation Inconsistency & Cardinality Errors	• Used identical Mermaid symbols for both mandatory (`||--o{`) and optional (`|o--o{`) participations. • Inconsistent relationship labels and cardinality reading directions (A → B).	• Added **Rule 7.4 (Symbol Matching)**: Cardinality symbols must perfectly reflect the participation text. • Added **Rule 4.4 (Notation Order)**: Enforced uniform A → B direction and established the markdown table as the authoritative source over the diagram.
3. Upstream Error Propagation & Invention	• Blindly inherited upstream errors from Agent 1 (e.g., duplicated `rejection_reason`, missing inference tags). • Invented unsupported attributes (e.g., `facility_description`).	• Added **Rule 3 & 3.1 (Upstream duplication detection)**: Instructed the agent to actively detect, filter out, and block duplicated attributes from prior outputs. • Mandated consistent carry-forward tagging for inferred elements.

Agent 3 — logical-database-designer (file 03) This version contained the most structural errors:

inconsistent in-row constraint enforcement, arbitrary nullability strengths, and many business rules without clear enforcement classification. (The key-standardization and foreign-key referential-action issues that also originated here are consolidated under Version 2, Issues 6 and 2, to avoid duplication.)

Table 3: Summary of Logical Designer Refinements

Issue Categories	**Examples in Version 1**	**Refinements (Agent/Prompt Changes)**
1. Constraint Enforcement & Nullability Flaws	• Omitted in-row constraints (e.g., temporal order `end > start` missing in some tables). • Inferred arbitrary `NOT NULL` strengths. • Failed to declare `UNIQUE` for candidate keys (e.g., `email`).	• Added **Rule 4**: add missing in-row temporal `CHECK`s and require `UNIQUE` for candidate keys. • Restricted nullability strength to strictly follow the source document requirements.
2. Enforcement Ambiguity & Upstream Inheritance	• Unclear boundaries on how complex rules should be enforced (e.g., relying solely on FKs for cross-table Role validation). • Silently inherited duplicated attributes (`facility_description`) from previous agents.	• Added **Rule 5 (Enforcement Classification)**: Required explicit classification of every business rule (e.g., distinguishing between in-table `CHECK` vs. cross-table Implementation Logic/Trigger). • Added **Rule 2 (Traceability Gate)**: Implemented strict guardrails to actively block specific upstream errors.

Agent 4 — database-design-reviewer (file 04) The Version 1 validation accurately cited sources but was overly lenient and missed errors: it contained internal contradictions, overly generous "Covered" ratings, and most critically, propagated technical errors from upstream instead of correcting them. It also missed exactly the structural issues that a reviewer role should have caught.

Table 4: Summary of Database Design Reviewer Refinements

Issue Categories	**Examples in Version 1**	**Refinements (Agent/Prompt Changes)**
1. Lack of Grading Discipline	• Inconsistent evaluation: contradictory ratings (e.g., "minor inconsistency" vs "reasonable") for the same issue. • Overly generous ratings for "Covered" rules despite missing implementation details.	• Added **Grading Discipline & Severity Rules**: Established strict rubrics that forbid "Grade A" if basic requirements are missing. • Mandated high-severity flags for systemic modeling gaps.
2. Evaluation Incompleteness	• Overlooked missing in-row `CHECK` constraints (e.g., temporal order). • Ignored constraint-strength mismatches (`NOT NULL` vs. source requirements). • Failed to flag incorrect/identical cardinality symbols.	• Expanded **Reviewer Checklist**: Added dedicated validation areas for simple in-row `CHECK`s, nullability strength, and precise cardinality symbol checks.
3. Traceability & Enforcement Gaps	• Propagated upstream incorrect premises (Mermaid errors) without correction. • Lacked a holistic view of business rule enforcement across layers (Analysis → DDL).	• Added **Business Rule Enforcement Matrix**: Required the reviewer to map every business rule across the Analysis, ERD, Logical, and DDL layers. • Added **Citation Rule**: Prohibited vague line-number citations and required justification tied to requirements.

Overall, the errors in Version 1 fell into four root cause categories. Each category was addressed by a corresponding type of change at the agent layer:

Summary — Summary of Root Cause Categories and Corresponding Changes - Version 1

Root Cause Category	Typical Manifestations	Type of Change in Agent
Over-inference beyond source	Fabricated usage policy rule; added `facility_description`; added outcome without label	Source-grounding rules, no-invented-elements, inference labeling
Duplication / lack of single source of truth	`rejection_reason` in two places	Single source of truth rule + upstream duplication detection
Inconsistent application	Temporal CHECKs, cardinality symbols, NOT NULL strength	Standardization rules (in-row CHECK, constraint-strength, notation order)
Missing scope / missing sections	Missing state transitions, role permissions, workflows, enforcement classification; reviewer omissions	Mandatory template sections + self-check checklists + grading discipline

Verification results in the subsequent version:

- file 01 is scope-clean and contains all required process sections;
- file 02 draws all relationships with correct symbols;
- file 03 has consistent in-row constraints and enforcement classification;
- file 04 applies disciplined scoring with a complete enforcement matrix and no longer propagates upstream errors.

2.4.2 Version 2

Context and Evaluation Methodology Version 2 again evaluated the outputs of the first four-agent pipeline (business analysis → conceptual design → logical design → validation) using the rubric-based method described in Section 2.2. Six issues were identified and traced back to the responsible agent's prompt/rules, following the same "fix the source" principle as Version 1.

Agent 3 — logical-database-designer (file 03) This stage produced the majority of structural defects in Version 2: missing referential-action policy, prose-only constraints, an incorrect cardinality enforcement, and inconsistent primary-key types across tables.

Table 5: Summary of Logical Designer Refinements

Issue Categories	Examples in Version 2	Refinements (Agent/Prompt Changes)
1. Missing Referential-Action Policy for FKs	• Foreign keys were defined without `ON DELETE` / `ON UPDATE` clauses. • No consistent basis for choosing `CASCADE`, `RESTRICT`, or `SET NULL`.	• Added a mandatory sub-rule to **Rule 3**: every FK must declare both actions. • Fixed decision criterion: `CASCADE` only for pure junction tables with no historical value; `RESTRICT/NO ACTION` as default for master/historical data; `SET NULL` rejected wherever it erases accountability.
2. Business Rules Left as Prose Instead of Named Constraints	• The rejection-reason rule (BR-14) was only described as "can be enforced by a CHECK" in prose, with no named constraint in the table definition.	• Added a sub-rule to **Rule 4**: every `CHECK`, `UNIQUE`, `FOREIGN KEY`, and `PRIMARY KEY` must be explicitly named (`CK_/UQ_/FK_/PK_`). • Every single-row conditional rule must become a named constraint (e.g. `CK_APPROVAL_DECISION_rejection_reason`); only genuine cross-row/cross-table rules remain as implementation logic.

Continued on next page

Table 5 – continued from previous page

Issue Categories	Examples in Version 2	Refinements (Agent/Prompt Changes)
3. Incorrect Cardinality Enforcement on Approval Decision	• A `UNIQUE` constraint was placed on `APPROVAL_DECISION.booking_id`, forcing a 1-to-1 relationship. • Conceptual ERD specifies `HAS_DECISION` as 1-to-0..*, so this silently discarded decision history.	• Removed the `UNIQUE` constraint by default. • Generalized the rule: a foreign key on the "many" side of a 1:0..* relationship must never carry `UNIQUE`, since this collapses the cardinality to 0..1 and loses history.
4. Inconsistent Primary-Key Types (Natural Keys Used Directly as PK)	• Natural identifiers were used directly as primary keys (e.g., student ID as `user_id`, space code as the `SPACE` PK). • Brittle under typos/reassignment; string-based FKs are less efficient for joins.	• Standardized every table to a surrogate `INT IDENTITY` primary key. • Natural identifiers demoted to ordinary attributes protected by a named `UNIQUE` constraint; all FKs rewritten to reference the surrogate `INT` keys. • Since surrogate keys are now immutable, `ON UPDATE` was unified to `NO ACTION` across all FKs, superseding the earlier cascade-on-update consideration from Issue 1.

Agents 1 & 4 — business-analyst (file 01) and database-design-reviewer (file 04) The reviewer had misclassified an application-layer gap as a data-model defect, blurring the boundary between database constraints and backend logic.

Table 6: Summary of Business Analyst / Reviewer Refinements

Issue Categories	Examples in Version 2	Refinements (Agent/Prompt Changes)
1. Application-Layer Gap Misclassified as a Schema Defect	• The reviewer flagged the missing `Cancelled` / `No-show` transition rules as a data-model defect. • Triggering a cancellation or no-show is actually an application-level status update that does not affect the two core workflows (booking, maintenance).	• Added **Rule 6.2** to business-analyst: classify these two transitions as an **Open Question** scoped to the application/business-workflow layer, not a design defect. • The values remain valid entries in the `status` domain.
2. No Self-Check Coverage for Prior Fixes	• The validation document had no checklist items covering the Cancelled/No-show classification, FK referential actions, named constraints, or the Approval Decision cardinality fix. • Later pipeline runs could not be automatically re-verified.	• Added **Required Validation Areas 18–21** to database-design-reviewer.md (one per issue). • Added corresponding checks to validation-rubric.md, so each fix is re-checked on every subsequent run.

Overall, the errors found in Version 2 fell into four root-cause categories, mirroring the pattern established in Version 1:

Summary — Summary of Root Cause Categories and Corresponding Changes - Version 2

Root Cause Category	Typical Manifestations	Type of Change in Agent
Scope confusion between database and application layers	Cancelled/No-show transitions treated as a schema defect	Explicit Open-Question classification rule + reviewer self-check
Missing enforcement policy	FKs without `ON DELETE`/`ON UPDATE`; conditional rules left as prose instead of named `CHECK` constraints	Mandatory referential-action criteria; mandatory named-constraint rule
Incorrect cardinality mapping	`UNIQUE` FK on Approval Decision collapsing 1:0..* to 1:1	Generalized rule: no `UNIQUE` on the "many" side of a 1:0..* relationship
Inconsistent key standardization	Mixed natural/surrogate primary keys across tables	Mandatory surrogate `INT IDENTITY` PK standardization rule

2.4.3 Version 3

In Version 3 we broadened the evaluation from data-model correctness to three further concerns: the *presentation* of the two entity-relationship diagrams, the *normalization* of controlled vocabularies into lookup tables, and the correct *scoping and traceability* of the implementation stage for Phase 1. As in the previous versions, we traced every defect to the responsible agent and fixed it at the prompt/rule level rather than in the generated `outputs/`.

Agent 2 & 3 — conceptual/logical designer (files 02, 03): diagram presentation The conceptual ERD (file 02) was drawn in crow's-foot (relational) notation, which visually duplicated the logical stage, while the logical design (file 03) contained no diagram at all — only textual relation definitions. In addition, both diagrams intermittently failed to render in Markdown because of Mermaid parse errors.

Table 7: Summary of Diagram-Presentation Refinements (Version 3)

Issue Categories	Examples in Version 3	Refinements (Agent/Prompt Changes)
1. Notation Mismatch Between Stages	• File 02 used crow's-foot (relational) notation, visually duplicating the logical stage. • File 03 had no diagram at all — only textual relation definitions.	• Rewrote the `conceptual-database-designer` workflow and **Rule 7** to produce a true Chen-notation ERD (rectangles, diamonds, ovals) via a Mermaid `flowchart`, split into one overview diagram plus one per-entity attribute diagram. • Added an Outputs-Format entry, a workflow step, and **Rule 6b** to `logical-database-designer` requiring a crow's-foot `erDiagram` of the relational schema.
2. Diagram Rendering Failures	• Edge/node labels containing parentheses, dots, or asterisks broke the `flowchart` parser. • An HTML underline tag was used for the key attribute. • The `erDiagram` declared attributes with combined key markers (`FK UK`) and multi-word data types.	• **Rule 7** now mandates wrapping every node/edge label in double quotes and marking the identifier with a `(PK)` suffix instead of an underline tag. • **Rule 6b** requires exactly one key marker per attribute, single-word data types, and one line per foreign-key relationship.

Agent 1, 2 & 3 — analyst/conceptual/logical (files 01–03): lookup-table normalization User role and the various status values were modelled as plain string attributes, and `department` was a free-text column on `USER_ACCOUNT`, which is brittle and not normalized.

Table 8: Summary of Lookup-Table Normalization Refinements (Version 3)

Issue Categories	Examples in Version 3	Refinements (Agent/Prompt Changes)
1. Free-Text Controlled Vocabularies	• `role` and status values (`account_status`, `space_status`, `booking_status`, `maintenance_status`) stored as plain strings. • `decision_outcome` risked a separate, redundant outcome table.	• Added **Rule 10** to `business-analyst` as a tagged design directive turning each vocabulary into a lookup entity. • Updated **Rule 4** of the conceptual and logical designers to realize closed vocabularies as lookup tables referenced by `INT` FKs (not `CHECK`). • `decision_outcome` reuses the shared `BOOKING_STATUS` lookup.
2. Denormalized Department	• `department` was a free-text column on `USER_ACCOUNT`.	• **Rule 10** promotes `DEPARTMENT` to its own entity (name plus a managing user). • The `DEPARTMENT` ↔ `USER_ACCOUNT` circular FK is resolved with a deferred `ALTER TABLE`; the reviewer and sample-data agents were updated to validate and seed the new lookup tables.

Agent 5 — database-definition-implementation-engineer (file 05): Phase-1 scoping and source fidelity At the start of Version 3 the implementation engineer over-reached into Phase 2 (it created trigger/stored-procedure stubs and foreign-key indexes, which produced errors), and, because of gaps left by upstream outputs, it silently self-corrected — inventing alternative constraint flows and renaming elements rather than implementing exactly what the design specified. Both behaviours were constrained at the agent level.

Table 9: Summary of Implementation-Engineer Refinements (Version 3)

Issue Categories	Examples in Version 2	Refinements (Agent/Prompt Changes)
1. Scope Overreach into Phase 2	• Phase 1 should cover only the basic schema (tables, keys, constraints), yet the engineer created trigger/stored-procedure stubs for business-rule logic and foreign-key indexes, which caused errors and exceeded scope.	• Changed **Rule 3** to defer every "requires implementation logic" rule to a `- PHASE 2` comment block (no triggers/procedures). • Added **Rule 5** forbidding manual indexes in Phase 1, and **Rule 7** recording the rejection-reason rule as a Phase 2 comment. • Added binding constraints to build only what the current Phase 1 requirement needs; sample-data and reviewer agents now treat these rules as "deferred to Phase 2," not defects.

Continued on next page

Table 9 – continued from previous page

Issue Categories	Examples in Version 2	Refinements (Agent/Prompt Changes)
2. Self-Correction & Hallucinated Constraints/Names	• Because upstream outputs had gaps, the engineer silently "fixed" them by inferring alternative constraint flows and different names — occasionally reasonable and even correct, but not what the validated design specified.	• Strengthened **Rule 1 (strict source fidelity)**: every table, column, type, nullability, and allowed value must trace to the logical design; inventing or "improving" elements is forbidden. • Strengthened **Rule 2 (exact constraint names)**: no renaming/abbreviating; deviations only for SQL Server name-length limits, and must be documented. • Added anti-hallucination guardrails: implement exactly what is validated and defer genuine gaps to `- OPEN QUESTION`, never silently resolving them.
3. Hard-Coded BR Numbers & Stale Table Order	• The engineer's rule tables hard-coded BR numbers offset from the analysis/logical numbering (e.g., rejection-reason cited as BR-13, check-in as BR-14, vs. BR-14 and BR-15 upstream), and assumed a stale eight-table order.	• Removed hard-coded BR numbers; **Rules 3 and 7** now cite BR numbers exactly as they appear in the logical design. • Updated **Rule 6** to the full fourteen-table creation order (lookup tables and `DEPARTMENT` first) so the DDL no longer self-justifies deviating from a stale list.

Overall, the Version 3 issues fell into four root-cause categories. Each category was addressed by a corresponding type of change at the agent layer:

Summary — Summary of Root Cause Categories and Corresponding Changes (Version 3)

Root Cause Category	Typical Manifestations	Type of Change in Agent
Inconsistent diagram presentation	Crow's-foot instead of Chen notation; missing logical diagram; Mermaid parse/render failures	Notation rules per stage (Chen vs crow's-foot) + strict label/marker syntax rules
Insufficient normalization	Role/status stored as free text; denormalized `department`	Lookup-table design directive; shared `BOOKING_STATUS`; `DEPARTMENT` entity with deferred FK
Scope overreach into Phase 2	Triggers/indexes pulled into a structure-only Phase 1 deliverable	Phase-2 deferral rules; no-trigger/no-index constraints; "build only what Phase 1 needs"
Implementation fidelity drift	Self-inferred constraints and renamed elements; hard-coded BR numbers offset from upstream	Anti-hallucination + exact-name rules; cite-BR-from-source rule; corrected table-creation order

Verification results after this version:

- file 02 renders a readable Chen-notation ERD (overview plus per-entity diagrams);
- file 03 includes a crow's-foot `erDiagram` that parses without error;
- files 01–05 model role/status/department as normalized lookup tables;
- file 05 is scoped to Phase 1 (no triggers/indexes), implements exactly the validated design, and cites BR numbers consistent with the upstream documents.

A note on the implementation stage (files 05–07)

Across all versions, the implementation artifacts — the DDL (file 05), the sample data (file 06), and the queries (file 07) — accumulated a relatively large number of individually reported issues. On closer inspection, however,

almost none of these were defects originating in the implementation agents themselves. They fell into two categories: (i) errors *inherited* from upstream design flaws (missing normalization, ambiguous cardinalities, unnamed constraints), which the implementation stage merely surfaced or, worse, tried to patch on its own; and (ii) complex enforcement structures — overlap checks, cross-table role restrictions, completion consistency — that genuinely belong to Phase 2 and cannot be expressed with plain `PK/FK/UNIQUE/CHECK`. Once the design was finalized upstream and the implementation agents were constrained to build only the Phase 1 structure (no triggers, no indexes) and to implement the validated design faithfully rather than self-correct, files 05–07 stabilized as well: the DDL deploys cleanly, the sample data satisfies every constraint, and the queries run against the implemented schema without modification. In other words, the quality of the implementation stage was almost entirely a function of the quality of the design that preceded it.

3 Conclusion

In this phase of the project, our group developed and refined an agent-based database design pipeline for the Campus Space Management System. Instead of manually fixing the generated outputs, we focused on improving the source of the outputs: the agent prompts, rules, templates, rubrics, and self-check instructions. This approach helped us maintain a clear and repeatable workflow, where each artifact could be regenerated from the same pipeline rather than corrected by hand.

Through three refinement versions, we observed that many database design problems were not isolated mistakes, but recurring patterns caused by unclear agent instructions. In Version 1, the main problems were hallucinated requirements, duplicated information, incorrect cardinalities, missing business-process sections, and weak reviewer discipline. In Version 2, we improved the handling of referential actions, named constraints, surrogate keys, cardinality enforcement. In Version 3, we further improved diagram notation, Mermaid rendering, lookup-table normalization, Phase 1 implementation scope, source fidelity, sample data consistency, and query design quality.

The most important lesson from this process is that an agent pipeline must be treated like a software system: errors should be traced to their root causes, converted into explicit rules, and verified through repeated execution. This is similar to debugging a program. Fixing only the final output may solve one visible error, but improving the agent rules prevents the same error from appearing again in future runs. As a result, the final pipeline became more consistent, more traceable, and more aligned with the original business requirements.

Overall, Phase 1 gave us practical experience in both database design and prompt-driven software engineering. We learned how to evaluate requirements, construct conceptual and logical database models, enforce database constraints, normalize repeated values into lookup tables, and separate database responsibilities from application-level logic. More importantly, we learned how human review and agent refinement can work together to produce better technical artifacts. The final result is not only a database design for the Campus Space Management System, but also a reusable improvement process for future agent-assisted database projects.